

Monte Carlo Options Pricing System

M5 — Risk Analytics — Design Checkpoint

Yassine

August 2026

1 Milestone Status

Milestone	Status	Tests
M1 — Data Ingestion	Complete	24
M2 — Calibration Engine	Complete	24
M3 — Simulation Engine	Complete	72
M4 — Options Pricing	Complete	87
M5 — Risk Analytics	Complete	162
Total		345¹

All 183 prior tests unchanged.

2 Architecture Overview

M5 adds three modules under `src/risk/` and a backward-compatible `z_extern` parameter to all simulators. No MLflow anywhere in the risk module — logging remains the caller’s responsibility.

```
src/risk/  
  metrics.py      # Pure functions on arrays  
  portfolio.py    # Correlation, Cholesky, portfolio simulation  
  scenarios.py    # Scenario model + presets  
  
src/simulation/  
  base.py         # z_extern on SimulatorBase  
  gbm.py          # z_extern support  
  jump_diffusion.py  
  heston.py
```

3 Design Decisions

3.1 Decision 1: Risk Metrics as Pure Functions

Risk metrics are stateless functions that receive arrays and return scalars. They know nothing about simulators, models, or pricing. The caller simulates first, then passes paths to the risk module. This keeps the module testable on any data source — simulated paths, historical data, or paths loaded from a file.

3.2 Decision 2: `z_extern` for Correlated Simulation

Rather than re-implementing path generation inside the portfolio module, existing simulators gained an optional `z_extern: np.ndarray | None` parameter. When provided, it supplies

the price-driving standard normals (shape: `n_sims × n_steps`). When `None`, each simulator generates its own internally — fully backward compatible.

How each model uses `z_extern`:

- **GBM**: `z_extern` is all of the randomness. `seed` becomes inert.
- **Jump-Diffusion**: `z_extern` replaces diffusion normals only. Poisson counts and jump sizes still draw from `seed`. Both parameters are live simultaneously.
- **Heston QE**: `z_extern` replaces the price-process normals. Variance-process uniforms still draw from `seed`. Both parameters are live simultaneously.

Constraint: `z_extern` and `variance_reduction` are mutually exclusive. They both control the random normals; using both raises `ValueError`.

3.3 Decision 3: Scenario Model with Preset Registry

Stress scenarios are `Scenario` Pydantic models with two override mechanisms:

- **Multipliers**: relative changes applied first (`{"sigma": 2.5}` means $\sigma \times 2.5$).
- **Overrides**: absolute values applied second and win over multipliers (`{"sigma": 0.8}` sets $\sigma = 0.8$).

Unknown keys in either dict raise `ValueError`, preventing silent typos. Presets are stored in a registry keyed by `(scenario_name, model_type)` since parameters differ across models — e.g., GBM has `sigma` while Heston has `v0` and `theta`.

Two preset families: `vol_spike` and `market_crash`, each with GBM, JD, and Heston variants. Rate shock was omitted because r is a pricing parameter (passed to `MCPricer.price()`), not a model parameter — it cannot be expressed as a model param override.

3.4 Decision 4: Correlation Estimation in Risk Module

Correlation matrix estimation lives in `src/risk/portfolio.py`, not in `src/calibration/`. It is tightly coupled to the multi-asset simulation workflow and is not model calibration in the M2 sense. Tickers are sorted alphabetically for deterministic matrix ordering.

3.5 Decision 5: VaR/CVaR Sign Convention

VaR and CVaR are returned as raw P&L percentiles: negative values mean losses. The presentation layer (API, frontend) handles sign flipping if desired. This avoids ambiguity and is consistent with the computation.

4 Risk Metrics — Definitions

4.1 Value at Risk (VaR)

The loss threshold at a given confidence level. The 95% VaR is the 5th percentile of the P&L distribution:

$$\text{VaR}_\alpha = F_{\text{P\&L}}^{-1}(1 - \alpha)$$

Interpretation: “with probability α , losses will not exceed $|\text{VaR}_\alpha|$.”

4.2 Conditional VaR (CVaR / Expected Shortfall)

The expected loss given that losses exceed VaR:

$$\text{CVaR}_\alpha = \mathbb{E}[\text{P\&L} \mid \text{P\&L} \leq \text{VaR}_\alpha]$$

Always $\leq$ VaR (more negative). Preferred by regulators because it captures the shape of the tail, not just a single percentile.

4.3 Max Drawdown

Worst peak-to-trough decline along a price path:

$$\text{MDD}_i = \max_{0 \leq s \leq t \leq T} \frac{S_i(s) - S_i(t)}{S_i(s)}$$

Computed via running maximum: `np.maximum.accumulate`. We report both the mean and 95th percentile across simulations, since the mean alone hides tail drawdown risk.

4.4 Sharpe Ratio

Risk-adjusted return:

$$\text{Sharpe} = \frac{\mathbb{E}[r] - r_f}{\sigma[r]}, \quad r_i = \frac{\text{P\&L}_i}{S_0}, \quad r_f = e^{rT} - 1$$

Denominator uses total return volatility with `ddof=1`.

4.5 Probability of Loss

Fraction of simulated paths ending below S_0 :

$$P(\text{loss}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[S_i(T) < S_0]$$

5 Multi-Asset Portfolio Simulation

5.1 Cholesky Decomposition

To simulate n correlated assets, we decompose the $n \times n$ correlation matrix Σ via Cholesky: $\Sigma = LL^\top$ where L is lower-triangular. Given independent standard normals $Z \in \mathbb{R}^n$, the product LZ yields correlated normals with the desired correlation structure.

5.2 Portfolio Path Construction

Each asset's paths are normalized to return space (starting at 1.0), then combined by portfolio weights:

$$P(t) = V_0 \sum_{i=1}^n w_i \cdot \frac{S_i(t)}{S_i(0)}$$

5.3 Seed Management

The master `seed` drives the independent normal generation and Cholesky application. Per-asset seeds for idiosyncratic randomness (JD jumps, Heston variance) are derived as `seed + i`, ensuring reproducibility while keeping idiosyncratic processes independent.

5.4 Positive Semi-Definite Fallback

At perfect correlation ($\rho = 1$), the matrix is singular and Cholesky fails. A ridge of $10^{-10} \cdot I$ is added as fallback. This injects $\sim 10^{-5}$ of independent noise per step, compounding to $\sim 2 \times 10^{-6}$ relative deviation at the terminal date. Tests at $\rho = 1$ assert at 10^{-4} tolerance with a comment naming the ridge as the cause.

6 Implementation Notes

6.1 JD Stress Test at Finer Timestep

The `market_crash` preset for jump-diffusion multiplies `lambda_j` by 5, which pushes $\lambda_j \cdot dt$ past the `MAX_LAMBDA_DT = 0.1` guard in the JD simulator. This is correct behavior — the single-jump-per-step approximation becomes invalid at high jump intensity. Tests use `dt/4` to stay within the guard.

API implication: at M6, the `/risk-metrics` endpoint with `market_crash/jump_diffusion` at daily steps ($dt = 1/252$) will raise unless the caller reduces `dt`. The endpoint should either auto-reduce `dt` or return a clear 422 explaining why.

6.2 Pydantic v2 Idiom

`model_config = ConfigDict(arbitrary_types_allowed=True)` used throughout instead of the deprecated inner class `Config`. Consistent with the v2 idiom already established in `src/config.py`.

6.3 Physical vs. Risk-Neutral Measure

Portfolio simulation in M5 operates under the **physical measure** (real-world drift from calibrated params). This is correct for risk analytics — VaR/CVaR measure real-world risk, not option-pricing risk. Risk-neutral portfolio simulation (for portfolio option pricing) would require passing $\mu = r - q$ via each asset’s simulator call; this is deferred.

6.4 Extra Test Coverage Beyond Spec

Tests added beyond the implementation brief:

- `z_extern` composing with `mu`: log-ratio of two runs differing only in drift equals exactly the drift gap (noise held fixed).
- Mixed GBM + JD + Heston portfolio: validates the `z_extern` architecture across heterogeneous models.
- Jointly-impossible 3×3 correlation matrix (pairwise plausible but not PSD): validates the eigenvalue check.
- Diversification lowers CVaR relative to $\rho = 1$: economic sanity check.

7 Cumulative System State

After M5, the system supports the full pipeline:

1. **M1:** Ingest OHLCV + options chains from yfinance into Postgres.
2. **M2:** Calibrate GBM, Jump-Diffusion, Heston from historical prices.
3. **M3:** Simulate price paths (vectorized, variance reduction, seeded).
4. **M4:** Price options (European, Asian, barrier, lookback), compute Greeks, benchmark against Black-Scholes.
5. **M5:** Compute VaR/CVaR/drawdown/Sharpe, simulate correlated multi-asset portfolios, stress test under preset scenarios.

On the horizon:

- M6 — FastAPI backend exposing all capabilities.
- Post-M4 Heston upgrade — calibrate against implied vol surface using options chain data (Heston characteristic function pricer + BS implied vol inversion).